

Agentic LMs: Hunting Down Test Smells

Rian Melo[‡], Pedro Simões[‡], Rohit Gheyi[‡], Marcelo d’Amorim^{||}, Márcio Ribeiro[†], Gustavo Soares[§], Eduardo Almeida[¶], Elvys Soares^{*}

[‡]UFCG, Brazil

Email: {rian.melo, pedro.henrique.lima.simoies}@ccc.ufcg.edu.br and rohit@dsc.ufcg.edu.br

^{||}NCSU, USA

Email: mdamori@ncsu.edu

[†]UFAL, Brazil

Email: marcio@ic.ufal.br

[§]Microsoft, USA

Email: gsoares@microsoft.com

[¶]UFBA, Brazil

Email: eduardo.almeida@ufba.br

^{*}IFAL, Brazil

Email: elvys.soares@ifal.edu.br



Abstract—Test smells reduce test suite reliability and complicate maintenance. While many methods detect test smells, few support automated removal, and most rely on static analysis or machine learning. This study evaluates models with relatively small parameter counts—Llama-3.2-3B, Gemma-2-9B, DeepSeek-R1-14B, and Phi-4-14B—for their ability to detect and refactor test smells using agent-based workflows. We assess workflows with one, two, and four agents over 150 instances of 5 common smells from real-world Java projects. Our approach generalizes to Python, Golang, and JavaScript. All models detected nearly all instances, with Phi-4-14B achieving the best refactoring accuracy (pass@5 of 75.3%). Phi-4-14B with four-agents performed within 5% of proprietary LLMs (single-agent). Multi-agent setups outperformed single-agent ones in three of five smell types, though for *Assertion Roulette*, one agent sufficed. We submitted pull requests with Phi-4-14B-generated code to open-source projects and six were merged.

1 INTRODUCTION

Test smells are design flaws in test code that compromise reliability and hinder maintenance [1]. They can reduce the effectiveness of test suites. These issues are prevalent in both open-source and industrial settings [2]. Common test smells include undocumented assertions (*Assertion Roulette*), complex conditional logic (*Conditional Test Logic*), duplicated assertions (*Duplicate Assert*), and hard-coded values (*Magic Number*) [2].

Existing tools for detecting test smells typically rely on static analysis [3] or machine learning [4], [3], but these approaches are often hard to adapt to new smells or other languages [3]. Recent advances in foundation models have opened new possibilities in software engineering, yet their use in test smell refactoring – especially through collaborative agentic workflows – remains underexplored.

In this work, we evaluate open models (Llama-3.2-3B, Gemma-2-9B, DeepSeek-R1-14B, and Phi-4-14B) for automatically detecting and refactoring test smells through agentic workflows. Our method supports one, two, or four agents and is easily extensible: new smells and refactorings can be defined in natural language. We evaluate 150 instances of 5 frequent test smells from real-world Java projects (Section 3).

Results show that all models detected nearly all test smell instances (pass@5 of 96% with four-agents), with Phi-4-14B achieving the highest refactoring accuracy (pass@5 of 75.3%). This performance is within 5% of proprietary LLMs using a single-agent. Multi-agent setups outperformed single-agent ones in three of five smell types, though for *Assertion Roulette*, a single-agent configuration proved more effective.

Preliminary results demonstrate promising generalization across programming languages – specifically Python, Golang, and JavaScript – using the same setup. Beyond automating the detection and refactoring of test smells, our approach also enables developers to interact with models to better understand the rationale behind suggested changes. All artifacts are publicly available online.¹

2 TEST SMELLS

Test smells are recurring patterns that reduce test clarity and maintainability. *Assertion Roulette* [1] occurs when a test method contains more than one assertion statement without an explanation or message (parameter in the assertion method). To mitigate this smell, developers should add a message to each assertion. *Conditional Test Logic* [5] arises when a test method contains one or more control statements

1. <https://zenodo.org/records/16740114>

(i.e., conditional expression, and loop statements). This can be addressed by having the developer create a dedicated test method for each condition. *Duplicate Assert* [2] appears when a test method contains more than one assertion statement with the same parameters. To address this, developers should split assertions that test different scenarios or states into separate tests for clarity. *Exception Handling* occurs when a test method contains either a throw statement or at least a catch clause. To avoid this smell, use the testing framework’s features (e.g., `assertThrows`) instead of manually catching or throwing exceptions. *Magic Number* occurs when a test method contains an assertion with a numeric literal as an argument. Refactoring involves extracting and initializing all magic numbers into constants or local variables with descriptive names.

3 METHODOLOGY

The primary goal of this study is to evaluate the effectiveness of an agentic approach using models to detect and remove test smells. We analyzed test methods from 11 real-world open-source GitHub projects using JUnit-5 previously studied by Soares et al. [6], including *janusgraph*, *quarkus*, *testcontainers-java*, *opengrok*, *jenkins*, *lettuce*, *Mindustry*, *data-transfer-project*, *Activiti*, *flowable-engine*, and *skywalking*. We limited our analysis to tests with up to 30 LOC, as 89% of test cases in the studied projects fall within this range. We focused on 5 common test smell types [2], selecting 30 real-world examples for each. We evaluate smaller models up to 14B parameters: Llama-3.2-3B, Gemma-2-9B, DeepSeek-R1-14B, and Phi-4-14B using their default configurations. All models were accessed via the Ollama platform and executed locally on a MacBook Pro M3 with 18GB of RAM (July 2025).

Agent communication is managed using the *LangChain API*, and we apply prompting techniques such as *Role (Persona)* and *Chain-of-Thought* [7] to enhance reasoning and contextual understanding. The four-agent configuration (Figure 1) distributes responsibilities as follows: Agent₁ detects potential smells; Agent₂ confirms the detection; Agent₃ performs the refactoring; and Agent₄ verifies the correctness of the refactored code. If there is disagreement, agents engage in an Evaluator-Optimizer loop², repeated up to three times.

Agent₁ utilizes the following prompt to detect test smells. The prompts use variables to represent relevant information required for each analysis. For instance, `test_smell_definition_and_refactoring` comprises two sentences: one that defines the smell and another that suggests an appropriate refactoring (see Section 2 for example sentences).

```
You are a coding assistant with many years of experience that
detects test smells.
test_smell_definition_and_refactoring
Your goal is to determine if the provided test code exhibits the test
smell "test_smell_name".
code
Next I may give you further details.
agent_feedback
If the test code contains test_smell_name, respond with EX-
ACTLY "YES" on the first line and explain why. Ignore code
```

```
comments. If it does not contain, say EXACTLY "NO" on the first
line and explain why not.
```

The prompt used by Agent₂ is designed to evaluate the output generated by Agent₁.

```
You are a coding expert reviewing the detection of a test smell.
Consider the following test smell.
test_smell_definition_and_refactoring
A previous agent analyzed the following test code.
code
It gave the following answer.
agent_answer
Your goal is to evaluate if the previous detection by another agent
is correct and justified. Ignore code comments. If you do not agree,
answer NO and explain what's wrong with it and what to correct.
If yes, just say YES.
```

To remove a test smell, Agent₃ utilizes the prompt outlined next. During the second or third iteration, it incorporates feedback from Agent₄.

```
You are a coding assistant specializing in test code analysis and
refactoring, with many years of experience.
test_smell_definition_and_refactoring
Your task is as follows. First analyze the provided test code to
resolve test smell occurrences "test_smell_name". If there is no
smell, output the original code unchanged. Second ensure the test
preserves the same behavior, but is free of test_smell_name.
Third output only the final refactored code, valid under JUnit 5.
Finally check the refactored version does not introduce compila-
tion errors.
Provide only the final refactored code, with no additional explana-
tion or text.
Code to analyze:
code
Next I may provide you further details.
agent_feedback
```

To evaluate the `refactored_code` proposed by Agent₃, Agent₄ utilizes the following prompt.

```
You are a code reviewer specializing in JUnit 5 test smells.
test_smell_definition_and_refactoring
Analyze the following code.
refactored_code
Your task is to check three conditions. First check the code does
not have the test smell test_smell_name. Second verify the
code follows JUnit 5 specification. Finally confirms the code does
not have compilation errors. If the code satisfy all conditions,
respond with EXACTLY "YES" on the first line. If not, respond
with EXACTLY "NO" on the first line, then explain in one or two
sentences why. Let's think step by step.
```

The two-agent setup follows the same workflow but omits Agents_{2,4}. Agent₁ handles detection, and Agent₃ performs the refactoring, without validation or feedback loops. In the single-agent setup, Agent₁ analyzes the test code, identifies potential smells, and applies the corresponding refactoring.

4 EVALUATION

Table 1 summarizes the detailed performance (pass@1) of the single-agent and multi-agent approaches for detecting and refactoring test smells.

4.1 Detecting Test Smells

Gemma-2-9B and Phi-4-14B exhibit good detection performance across most test smells, correctly identifying 98% and 96% instances, respectively, when using two agents.

2. <https://www.anthropic.com/research/building-effective-agents>

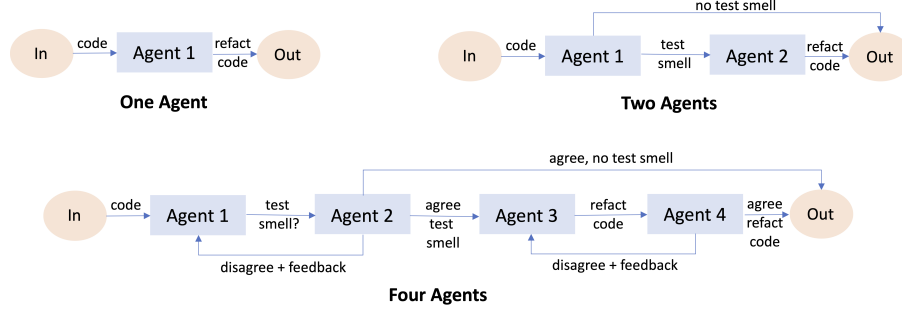


Figure 1: Agentic workflow with one, two and four agents to detect and remove test smells.

Table 1: Detection and refactoring performance (pass@1) of test smells by one, two, and four agents using Llama-3.2-3B (Lla), Gemma-2-9B (Gem), DeepSeek-R1-14B (DS) and Phi-4-14B (Phi).

		One Agent					Two Agents								Four Agents							
		Detect and Refactoring					Detect				Refactoring				Detect				Refactoring			
	Sub.	Phi	DS	Gem	Lla		Phi	DS	Gem	Lla	Phi	DS	Gem	Lla	Phi	DS	Gem	Lla	Phi	DS	Gem	Lla
Assertion Roulette	30	93.3%	56.7%	66.7%	20.0%		100.0%	86.7%	96.7%	100.0%	70.0%	33.3%	36.7%	26.7%	100.0%	86.7%	96.7%	100.0%	73.3%	33.3%	36.7%	26.7%
Cond. Test Logic	30	3.3%	6.7%	6.7%	0.0%		96.7%	93.3%	100.0%	100.0%	13.3%	0.0%	13.3%	0.0%	96.7%	93.3%	100.0%	100.0%	10.0%	3.3%	16.7%	3.3%
Duplicate Assert	30	40.0%	36.7%	23.3%	10.0%		100.0%	50.0%	100.0%	100.0%	70.0%	20.0%	56.7%	10.0%	93.3%	60.0%	100.0%	100.0%	60.0%	20.0%	60.0%	10.0%
Exception Handling	30	43.3%	13.3%	40.0%	36.7%		90.0%	50.0%	93.3%	100.0%	43.3%	23.3%	43.3%	30.0%	86.7%	50.0%	90.0%	100.0%	40.0%	23.3%	40.0%	36.7%
Magic Number	30	73.3%	83.3%	30.0%	33.3%		93.3%	100.0%	100.0%	100.0%	80.0%	90.0%	43.3%	40.0%	90.0%	100.0%	100.0%	100.0%	70.0%	90.0%	50.0%	33.3%

Their performance remains consistent with one or four agents. For every test smell, at least one model achieves a perfect detection score. DeepSeek-R1-14B demonstrates the weakest performance, detecting only 78% instances. In contrast, Llama-3.2-3B consistently detects all test smells across all agent configurations.

We review Agents_{1,2} outputs to assess the validity of their test smell explanations. Next, Agent₁ uses Phi-4-14B to identify the *Exception Handling* test smell. The other models also detect it.

"The test method testStackBlowOut includes a try-catch block to handle exceptions manually, which is indicative of the Exception Handling test smell. The code attempts to catch an IllegalArgumentException, rather than using testing framework features like assertThrows to verify that the exception is thrown as expected."

Llama-3.2-3B achieved the highest detection rate, identifying all test smell instances. Gemma-2-9B and Phi-4-14B missed a few but provided more detailed and accurate explanations. These results demonstrate that smaller models can reliably detect test smells when given concise natural language definitions. Unlike static-analysis tools, models take a semantic approach, recognizing frameworks like Mockito or JUnit without needing exhaustive method lists.

4.2 Refactoring Test Smells

The optimal configuration for Llama-3.2-3B and Gemma-2-9B involved using four-agents, successfully refactoring 22% and 40.7% out of 150 instances in one attempt, respectively. DeepSeek-R1-14B achieved its best performance with a single-agent, correctly refactoring 39.3% instances. In contrast, Phi-4-14B performed best with two-agents, successfully refactoring 55.3% instances, demonstrating its effectiveness in a multi-agent setup. Processing 150 smells using four-agents took ~1h (Llama-3.2-3B, Gemma-2-9B), ~5h (Phi-4-14B), and ~8h (DeepSeek-R1-14B).

Using single, two, or four-agents with Llama-3.2-3B, Gemma-2-9B, or Phi-4-14B, correctly remove the *Exception*

Handling test smell. For instance, they produce the code shown in Listing 1, where the try-catch block is removed, and the fail statement is replaced with an `assertThrows` statement.

Listing 1: Correct refactoring for removing *Exception Handling* using single and multi-agents with Gemma-2-9B.

```
@Test
public void testStackBlowOut() {
    final SmallRyeConfig config =
        ↪ buildConfig(maps(singletonMap("foo.blowout",
        ↪ "${foo.blowout}")));
    assertThrows(IllegalArgumentException.class, () ->
        ↪ config.getValue("foo.blowout", String.class));
}
```

In some cases, using single-agent with Gemma-2-9B to refactor results in incorrect code that alters the program's behavior by adding an extra assertion when removing the *Conditional Test Logic* test smell. In contrast, using multi-agents produces a correct refactoring, effectively separating the logic into two methods.

We can also correctly apply a refactoring to remove *Duplicate Assert* test smells using multi-agents with Phi-4-14B. However, using a single-agent with Phi-4-14B applies a refactoring strategy that differs from the one proposed for this smell. Specifically, it incorporates a unique message for each assertion, effectively eliminating both this test smell and the *Assertion Roulette* test smell.

DeepSeek-R1-14B provides significantly more details about its reasoning process compared to the other models. In some cases, it initially proposes a valid refactored code or a correct refactoring strategy during its thought process but later revises its approach, ultimately leading to an incorrect solution.

For the *Assertion Roulette* test smell, Phi-4-14B achieved the best result, correctly detecting and refactoring 93.3% of instances with a single-agent. In contrast, all models struggled with *Conditional Test Logic*, where we evaluated only one of several possible refactoring strategies. For *Duplicate Assert*, the highest success rate (70%) was obtained by Phi-

4-14B using two-agents. Phi-4-14B also performed best on *Exception Handling*, correctly refactoring 43.3% of cases with a single-agent. The best results for *Magic Number* came from DeepSeek-R1-14B with two agents, achieving 90% accuracy. Overall, two or four-agent setups led to better outcomes in three of the five test smells. However, for *Assertion Roulette* and *Exception Handling*, a single-agent was sufficient. Notably, there are 23 unique instances successfully refactored by at least one model – Llama-3.2-3B (6), Gemma-2-9B (14), or DeepSeek-R1-14B (6) – that Phi-4-14B with two-agents failed to refactor. By combining strengths across models, we achieve correct refactorings for 70.6% instances (pass@1).

To improve performance under the default configuration, we conducted an experiment to assess the impact of temperature [8] settings on Phi-4-14B’s ability to detect and refactor test smells. A temperature of 0.9 yielded the best results for both tasks, slightly improving performance by 3.3%.

4.3 Failure Types

The ablation study revealed distinct failure patterns across agent configurations. In the single-agent setup, issues frequently included incomplete fixes, superficial code changes that failed to resolve the smell, and the introduction of invalid or non-existent constructs—often stemming from incorrect syntax or reliance on frameworks other than JUnit-5. In *Magic Number*, the model sometimes extracted only a subset of constants or failed to initialize new variables. Refactorings for *Conditional Test Logic* and *Duplicate Assert* occasionally introduced subtle logic changes. In *Exception Handling*, some transformations mistakenly removed critical code alongside the `try` block.

Introducing additional agents reduced some errors but brought new challenges, including increased code divergence, missed detections that the single agent previously succeeded on, and frequent disagreements—particularly with Agent₄ rejecting correct refactorings proposed by Agent₃. Moreover, if the multi-agent setup had used JUnit-5 instead of JUnit-4 and correctly positioned the message parameter, its performance in *Assertion Roulette* would have matched that of the single-agent configuration.

4.4 Feedback Loop

The feedback loop was triggered when Agents_{2,4} disagreed with Agents_{1,3}, respectively (see Figure 1). For detecting test smells, Gemma-2-9B and Phi-4-14B utilized the feedback loop in 1 and 9 test smell instances, respectively. In 2 instances, Phi-4-14B was able to correctly identify a test smell due to the feedback loop. Llama-3.2-3B did not require the feedback loop for detecting any test smells. DeepSeek-R1-14B successfully detected 3 instances of the *Duplicate Assert* test smell following a feedback loop. We employed a three-iteration feedback loop, which was sufficient for Agents_{1,2} to reach a consensus on detecting the majority of test smells. For Agents_{3,4} to reach a consensus on refactoring test smells, only 7.3% of the cases required more than three iterations. By allowing additional iterations, we achieved consensus in all cases by the eighth iteration, except for a single instance of the *Conditional Test Logic* test smell.

4.5 Pull Requests

To assess the acceptance of the refactorings generated by Phi-4-14B, which achieved the best performance in our study, we submitted 10 pull requests, ensuring at least one for each test smell type across five different open-source projects. As of this writing, six of these pull requests have been reviewed and merged by human maintainers, who deemed them valuable: two in *janusgraph* (*Assertion Roulette* and *Exception Handling*), two in *lettuce* (*Magic Number* and *Duplicate Assert*), and one each in *opengrok* and *jenkins* (both addressing *Duplicate Assert*). Two pull requests were rejected because contributors noted that the additional message in each assertion did not improve their code. Two pull requests remain open.

4.6 LLMs

In July 2025, we also run o3, Claude-4-Sonnet, and Gemini-2.5-Pro using a single-agent setup on 150 test smells. Claude-4-Sonnet, o3, Gemini-2.5-Pro achieved a refactoring performance (pass@1) of 59.3%, 74.7%, and 72.0%, respectively—though at significantly higher computational cost.

We allowed Phi-4-14B with four-agents to make up to five attempts, rather than limiting it to just one. With pass@5 [9], it successfully detected 96% and refactored 75.3% of the test smell instances. These results indicate that the open Phi-4-14B, when paired with agentic workflows, can rival proprietary state-of-the-art LLMs while remaining cost-effective and executable on consumer-grade hardware.

4.7 Prompts

We refined the prompts iteratively. For instance, in the definition of *Assertion Roulette*, the phrase “has more than one assertion” proved to be more precise and effective than the commonly used “has multiple non-documented assertion” found in the literature [2]. Early experiments overlooked prompt minimization, resulting in suboptimal outcomes – consistent with Hsieh et al. [10]. Providing Agent₄ with the original and refactored code plus Agent₃’s full explanation overloaded the model and reduced its effectiveness. Similarly, overly detailed smell definitions negatively impacted performance. Although not formally tested, we observed that large prompts hindered reasoning. Streamlining definitions and instructing Agent₄ to respond concisely improved efficiency. These findings align with prior studies showing that excessive or irrelevant context degrades LLM performance [11]. In some cases, we found that sentence order influenced model reasoning, supporting prior work [12].

4.8 Other Languages

We assess whether the same setup can detect and refactor test smells across different programming languages. Using a single-agent configuration, Phi-4-14B successfully refactored (pass@1) 24/30, 27/30, and 29/30 test smells in Golang, JavaScript, and Python, respectively. These instances contained *Assertion Roulette*, *Duplicate Assert*, and *Magic Number* smells in real-world projects such as *kubernetes*, *react-beautiful-dnd*, and *django*. The model achieved performance comparable to that observed with Java test cases. In the JavaScript setting, it occasionally generated Java code; however, when

explicitly instructed to produce JavaScript test code, it responded correctly. JUnit-5 compliance can also be excluded from the prompts. These preliminary results suggest that the proposed approach is promising for detecting and refactoring test smells across a variety of programming languages.

4.9 Detecting Other Types of Test Smells

We evaluated Phi-4-14B on 30 types of test smells [15], using real-world examples—18 in Java and 12 in other languages including Ruby, Python, JavaScript, Smalltalk, and TTCN-3. A single-agent was used, focusing solely on detection, as the catalog does not define refactoring strategies for all smells. Phi-4-14B successfully detected 28 out of 30 test smells (pass@1), providing detailed justifications for each. The two initial failures were due to ambiguous definitions, which, once clarified, enabled the accurate detection of all smells. We also evaluated Phi-4-14B on 45 instances spanning four additional test-smell types in JavaScript. It misclassified only four cases (pass@1) of *OverCommented Tests*, due to an imprecise definition of when a test contains excessive comments. After clarifying the definition, it correctly detected all test smells.

4.10 Larger Test Cases

As previously noted, 89% of the test cases in the 11 open-source projects analyzed contain at most 30 lines of code (LOC). To further evaluate Phi-4-14B, which demonstrated the best performance in our study, we selected 30 test smells (five per smell type) exceeding 30 LOC from these 11 open-source projects. We employed the same four-agent setup to detect and refactor these test smells. Although the strategy successfully identified 86% of the test smells, it showed a decline in performance compared to previous results. The effectiveness in accurately refactoring test smells also diminished, with a pass@1 rate of 28%.

4.11 Threats to Validity

One potential threat to internal validity is data leakage, where test smells analyzed in this study may be part of the foundation models’ training data. To mitigate this, we applied Metamorphic Testing.¹ We limit our analysis to test code with a maximum of 30 LOC. While this constraint excludes longer test cases, a substantial number of test cases still fall within this criterion, ensuring a representative evaluation. Another factor is prompt design, which can lead to generic responses instead of targeted explanations. To minimize variability, we used concise prompts across all evaluations, which were carefully reviewed by three authors of this paper. Validating alignment between the model output and test smell definitions is also challenging. To ensure accuracy, three authors independently reviewed selected responses, verifying whether the refactored code adhered to the intended definitions. Construct validity is limited by the small number of examples for each test smell, though they are real cases from GitHub repositories.

5 RELATED WORK

Aljedaani et al. [3] compiled a comprehensive catalog of 22 test smell detection tools, the majority of which are designed

for Java, SmallTalk, C++, and Scala, and 4 refactoring tools. Soares et al. [6] conducted a mixed-methods analysis involving 485 Java projects, exploring the adoption of JUnit-5 features to improve test code quality. Wang et al. [13] propose a tool called PyNose to detect 18 types of test smells in Python. Lambiase et al. [14] introduce DARTS, an IntelliJ plugin that detects and refactors three test smells: *Eager Test*, *General Fixture*, and *Lack of Cohesion of Test Methods*. Pontillo et al. [4] proposed a machine learning-based approach to detect test smells, focusing on four specific types. Lucas et al. [15] investigated the capability of three LLMs to detect test smells across multiple programming languages.

We investigate the effectiveness of agentic workflows for detecting and refactoring test smells in Java test cases. Our approach coordinates up to four-agents to automate these tasks. Results show that Phi-4-14B achieves strong refactoring performance. It runs efficiently on consumer-grade hardware and perform comparably to LLMs. A key advantage of our approach is its extensibility: new smells can be added via natural language definitions. Preliminary experiments also confirm successful application to Python, Golang, and JavaScript, highlighting its cross-language potential. Unlike conventional methods that only detect smells or suggest code changes, our approach supports interactive conversations and explanatory feedback.

6 CONCLUSIONS

We evaluated Llama-3.2-3B, Gemma-2-9B, DeepSeek-R1-14B, and Phi-4-14B for the automated detection and refactoring of test smells using agentic workflows with **one, two, and four agents**. The evaluation covered 150 instances of 5 common test smell types from real-world projects. We assessed each model’s ability to perform **automated refactorings**. Phi-4-14B, Gemma-2-9B, and Llama-3.2-3B successfully detected nearly all test smell instances. Phi-4-14B delivered the best refactoring performance, reaching a pass@5 of 75.3%, and performed within 5% of proprietary models like o3, Claude-4-Sonnet, and Gemini-2.5-Pro using a single-agent. We submitted **10 pull requests** with code refactored by Phi-4-14B. Six were merged, two was rejected, and two remain open – demonstrating the practical utility of our approach in live development environments. **Multi-agent workflows outperformed single-agent setups** in three out of five test smell types, showing their potential to enhance code quality with minimal developer effort.

REFERENCES

- [1] A. van Deursen, L. Moonen, A. van Den Bergh, and G. Kok, “Refactoring test code,” in *XP*, 2001, pp. 92–95.
- [2] A. Peruma, K. Almalki, C. D. Newman, M. W. Mkaouer, A. Ouni, and F. Palomba, “On the distribution of test smells in open source Android applications: An exploratory study,” in *CSSE*, 2019, pp. 193–202.
- [3] W. Aljedaani et al., “Test smell detection tools: A systematic mapping study,” in *EASE*, 2021, pp. 170–180.
- [4] V. Pontillo, D. Amoroso d’Aragona, F. Pecorelli, D. Di Nucci, F. Ferrucci, and F. Palomba, “Machine learning-based test smell detection,” *Empirical Software Engineering*, vol. 29, no. 2, pp. 1–44, 2024.
- [5] G. Meszaros, *xUnit test patterns: Refactoring test code*. Pearson, 2007.

- [6] E. Soares, M. Ribeiro, R. Gheyi, G. Amaral, and A. L. M. Santos, "Refactoring test smells with JUnit 5: Why should developers keep up-to-date?" *TSE*, vol. 49, no. 3, pp. 1152–1170, 2023.
- [7] S. Schulhoff et al., "The prompt report: A systematic survey of prompting techniques," 2024. <https://arxiv.org/abs/2406.06608>
- [8] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi, "The curious case of neural text degeneration," in *ICLR*, 2020.
- [9] M. Chen et al., "Evaluating LLMs trained on code," 2021. <https://arxiv.org/abs/2107.03374>
- [10] C.-P. Hsieh et al., "RULER: What's the real context size of your long-context language models?" 2024. <https://arxiv.org/abs/2404.06654>
- [11] F. Shi et al., "LLMs can be easily distracted by irrelevant context," in *ICML*, vol. 202. 2023, pp. 31 210–31 227.
- [12] X. Chen, R. A. Chi, X. Wang, and D. Zhou, "Premise order matters in reasoning with LLMs," in *ICML*. 2024.
- [13] T. Wang, Y. Golubev, O. Smirnov, J. Li, T. Bryksin, and I. Ahmed, "PyNose: A test smell detector for Python," in *ASE*. 2021, pp. 593–605.
- [14] S. Lambiase, A. Cupito, F. Pecorelli, A. De Lucia, and F. Palomba, "Just-in-time test smell detection and refactoring: The DARTS project," in *ICPC*, 2020, pp. 441–445.
- [15] K. Lucas, R. Gheyi, E. Soares, M. Ribeiro, and I. Machado, "Evaluating large language models in detecting test smells," in *SBES*, 2024, pp. 672–678.